

Homework 06

Question 1: Compare KNN and logistic regression

Use [breast-cancer.csv](#), which contains 569 cases from the [Wisconsin Diagnostic Breast Cancer dataset](#). Predict `y` (1 for malignant and 0 for benign) using `mean_radius`, `mean_texture`, and `mean_smoothness`.

Set a random seed and randomly split the observations into 70% training and 30% test data. Use the same split for every model. Standardize the predictors using the training means and standard deviations, and apply the same transformations to the test data.

Fit ordinary logistic regression with an intercept and no penalty, and KNN with Euclidean distance, equal neighbor weights, and

$$k \in \{5, 15, 25, 35, 45\}.$$

Using the estimated probabilities for `y = 1` on the test data, plot all six ROC curves in one figure and report their AUC values. Which of the six fitted models do you prefer? Give a brief justification based on your results.

ANSWER: We compare the models using their estimated probabilities of malignancy, not class predictions at a single cutoff. For KNN, this probability is the proportion of malignant cases among the k nearest training observations:

$$\hat{p}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i,$$

where $N_k(\mathbf{x})$ is the set of those neighbors. Averaging the numeric responses 0 and 1 gives this probability directly.

First split the data, then estimate each predictor's mean and standard deviation from the training observations. Use those same values to standardize both sets. This prevents the test observations from influencing the fitted distance scale.

```
library(FNN)
library(pROC)
```

Type `'citation("pROC")'` for a citation.

Attaching package: 'pROC'

The following objects are masked from 'package:stats':

`cov`, `smooth`, `var`

```
data_dir <- if (file.exists("data/breast-cancer.csv")) {
  "data"
} else {
  "practice/weeks/week-06/data"
}
data <- read.csv(file.path(data_dir, "breast-cancer.csv"))
set.seed(432)
train_id <- sort(sample(seq_len(nrow(data)), floor(0.7 * nrow(data))))
train <- data[train_id, ]
test <- data[-train_id, ]
predictors <- c("mean_radius", "mean_texture", "mean_smoothness")

x_bar <- colMeans(train[, predictors])
s <- sapply(train[, predictors], sd)
X_train <- scale(train[, predictors], center = x_bar, scale = s)
X_test <- scale(test[, predictors], center = x_bar, scale = s)
y_train <- train$y
y_test <- test$y
```

Fit ordinary logistic regression and the five KNN models. In R, `knn.reg` averages the 0/1 responses; in Python, `predict_proba` returns the same neighbor proportions. Logistic regression includes an intercept.

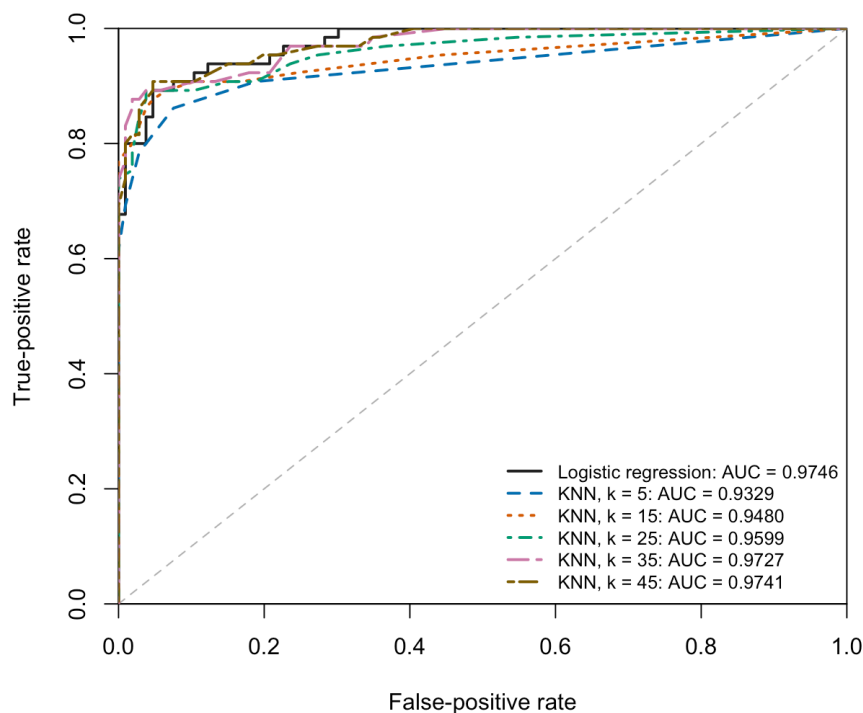
```
fit <- glm(y ~ ., data = data.frame(y = y_train, X_train),
  family = binomial())
k_values <- c(5, 15, 25, 35, 45)
model_names <- c("Logistic regression", paste("KNN, k =", k_values))
p_test <- matrix(NA_real_, nrow(X_test), length(model_names))
p_test[, 1] <- predict(fit, newdata = data.frame(X_test), type = "response")
for (m in seq_along(k_values)) {
  p_test[, m + 1] <- knn.reg(train = X_train, test = X_test,
    y = y_train, k = k_values[m],
    algorithm = "brute")$pred
}

roc_test <- vector("list", length(model_names))
auc_test <- numeric(length(model_names))
for (m in seq_along(model_names)) {
```

```
roc_test[[m]] <- roc(y_test, p_test[, m], levels = c(0, 1),
                    direction = "<", quiet = TRUE)
auc_test[m] <- as.numeric(auc(roc_test[[m]]))
}
data.frame(Model = model_names, Test_AUC = round(auc_test, 4))
```

	Model	Test_AUC
1	Logistic regression	0.9746
2	KNN, k = 5	0.9329
3	KNN, k = 15	0.9480
4	KNN, k = 25	0.9599
5	KNN, k = 35	0.9727
6	KNN, k = 45	0.9741

```
colors <- c("#222222", "#0072B2", "#D55E00", "#009E73", "#CC79A7", "#806000")
line_types <- c(1, 2, 3, 4, 5, 6)
plot(c(0, 1), c(0, 1), type = "n", xaxs = "i", yaxs = "i",
     xlab = "False-positive rate", ylab = "True-positive rate")
abline(0, 1, col = "gray70", lty = 2)
for (m in seq_along(model_names)) {
  lines(1 - roc_test[[m]]$specificities, roc_test[[m]]$sensitivities,
       col = colors[m], lty = line_types[m], lwd = 2)
}
legend("bottomright", sprintf("%s: AUC = %.4f", model_names, auc_test),
     col = colors, lty = line_types, lwd = 2, cex = 0.8, bty = "n")
```



Test ROC curves for logistic regression and five KNN fits on the same 171 observations.

For this split, I prefer logistic regression. Its test AUC is 0.9746, the largest of the six values, and it gives a simple fitted model. KNN with $k = 45$ is nearly tied at 0.9741, followed by $k = 35$ at 0.9727. The smaller neighborhoods have lower test AUCs: 0.9329, 0.9480, and 0.9599 for $k = 5, 15, 25$, respectively. The ROC curves show that the larger neighborhoods rank these test cases more effectively than $k = 5$ does.

This is a comparison on one test split. A small AUC difference does not establish that one method is always better, and AUC measures ranking rather than the fraction of correct class predictions. For a final model-selection exercise, choose the method and k using validation or cross-validation within the training data, then evaluate the chosen model on a fresh test set.

Question 2: Training and test ROC curves

Use the same data, training/test split, and standardization as in Question 1. Fit KNN with Euclidean distance, equal neighbor weights, and

$$k \in \{1, 3, 5, 10, 20\}.$$

Use each fitted model to estimate probabilities for both the training and test observations. For training predictions, include each observation itself among its k nearest neighbors.

For each k , use a separate panel to show the training and test ROC curves together. Report both AUC values.

Explain the difference between training and test performance and how it changes with k . In particular, explain what happens at $k = 1$ and why training performance may not reflect performance on new observations.

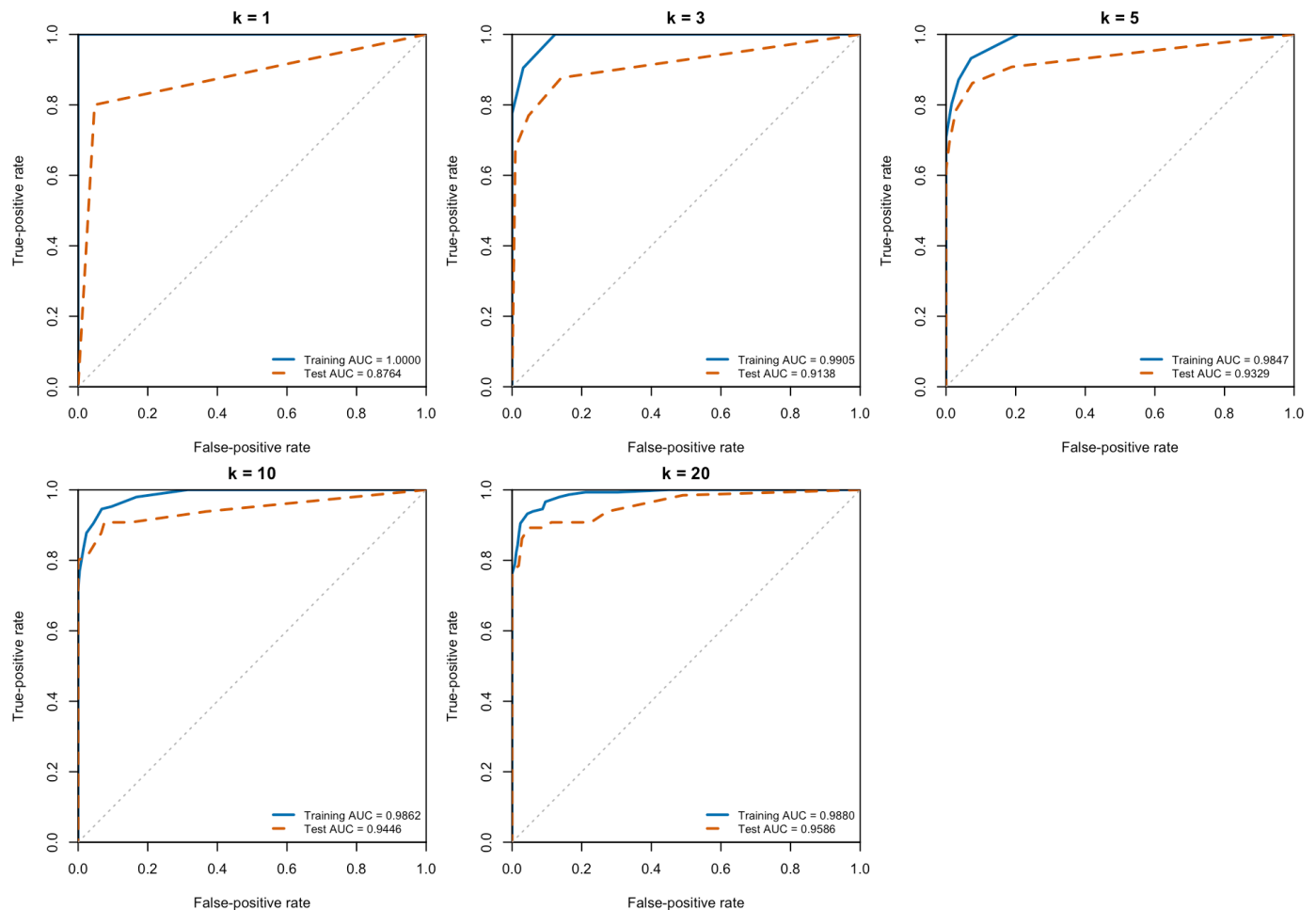
ANSWER: At $k = 1$, each training observation is its own nearest neighbor. Thus its fitted probability equals its observed response, giving a training AUC of 1. This perfect training ranking does not establish good prediction for a new observation, whose label was not used in fitting.

Reuse the data split and standardization from Question 1. In particular, supply the training predictors explicitly when requesting training predictions so that each observation can be its own neighbor.

```
k_values <- c(1, 3, 5, 10, 20)
p_train <- matrix(NA_real_, nrow(X_train), length(k_values))
p_test <- matrix(NA_real_, nrow(X_test), length(k_values))
roc_train <- roc_test <- vector("list", length(k_values))
auc_train <- auc_test <- numeric(length(k_values))
for (m in seq_along(k_values)) {
  k <- k_values[m]
  # test = X_train includes each training observation itself.
  p_train[, m] <- knn.reg(train = X_train, test = X_train,
                        y = y_train, k = k, algorithm = "brute")$pred
  p_test[, m] <- knn.reg(train = X_train, test = X_test,
                        y = y_train, k = k, algorithm = "brute")$pred
  roc_train[[m]] <- roc(y_train, p_train[, m], levels = c(0, 1),
                      direction = "<", quiet = TRUE)
  roc_test[[m]] <- roc(y_test, p_test[, m], levels = c(0, 1),
                     direction = "<", quiet = TRUE)
  auc_train[m] <- as.numeric(auc(roc_train[[m]]))
  auc_test[m] <- as.numeric(auc(roc_test[[m]]))
}
data.frame(k = k_values, Training_AUC = round(auc_train, 4),
           Test_AUC = round(auc_test, 4))
```

	k	Training_AUC	Test_AUC
1	1	1.0000	0.8764
2	3	0.9905	0.9138
3	5	0.9847	0.9329
4	10	0.9862	0.9446
5	20	0.9880	0.9586

```
old_par <- par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))
for (m in seq_along(k_values)) {
  plot(1 - roc_train[[m]]$specificities, roc_train[[m]]$sensitivities,
      type = "l", col = "#0072B2", lwd = 2,
      xlim = c(0, 1), ylim = c(0, 1), xaxs = "i", yaxs = "i",
      main = paste("k =", k_values[m]),
      xlab = "False-positive rate", ylab = "True-positive rate")
  lines(1 - roc_test[[m]]$specificities, roc_test[[m]]$sensitivities,
      col = "#D55E00", lty = 2, lwd = 2)
  abline(0, 1, col = "gray70", lty = 3)
  legend("bottomright", c(sprintf("Training AUC = %.4f", auc_train[m]),
                              sprintf("Test AUC = %.4f", auc_test[m])),
      col = c("#0072B2", "#D55E00"), lty = c(1, 2), lwd = 2,
      cex = 0.8, bty = "n")
}
plot.new()
```



Training and test ROC curves for each KNN neighborhood size. Each training observation is included among its own neighbors.

```
par(old_par)
```

The training AUC is 1.0000 at $k = 1$, whereas the test AUC is only 0.8764. As k increases through 3, 5, 10, 20, test AUC rises to 0.9138, 0.9329, 0.9446, and 0.9586. The training-test gap shrinks from about 0.124 at $k = 1$ to 0.029 at $k = 20$.

Training AUC does not decrease at every step: it falls to 0.9847 at $k = 5$, then rises to 0.9862 at $k = 10$ and 0.9880 at $k = 20$. Although larger neighborhoods smooth the fitted probabilities, their ranking can improve on both sets. The important pattern here is the overly optimistic training performance for very small k , especially $k = 1$.

Increasing k reduces the influence of any one training label. It often reduces the optimism of training performance and makes the estimated probabilities less variable. If k becomes too large, however, averaging over distant observations can miss local structure. Neither test AUC nor the training-test gap must change monotonically on a particular split.

For 1NN, the estimated probabilities take only the values 0 and 1. Its test ROC curve therefore has one nontrivial point, and its AUC equals (sensitivity + specificity)/2. The perfect training AUC is a consequence of reusing each observation's own label; the test curve evaluates predictions for observations outside the training set.

Question 3: Peer evaluation of Homework 5

You will receive two Homework 5 submissions to review. The instructor's GitHub account, **rqzhu-aide**, will open a **GitHub Issue** in the repository containing your Homework 5 submission. It will contain links to two other students' repositories, where you can find their submitted homework reports.

Open your repository's **Issues** tab on GitHub to find the assignment. You may ask AI to explain GitHub Issues or help you locate the assignment.

Choose **one specific mistake or item for improvement** from any question in either report.

- 1. Identify the mistake or item for improvement.** State which report and question you selected, and briefly describe what is wrong or could be improved. Examples include incorrect code, a calculation error, a misinterpretation of results, a plot that could communicate the findings more clearly, or anything else you think could be done better.
- 2. Explain your assessment.** Quote the relevant text, code, or figure caption exactly. Explain why the selected answer is incorrect or how it could be improved, referring to its reasoning, calculations, code, figures, or interpretation. **Complete this part yourself, without AI assistance.**
- 3. Provide a correction.** Write a corrected or improved answer to the selected question, including any calculations, code, or figures needed.

After completing your review, **open a GitHub Issue in the repository containing the report you selected**. Identify the report and question, and include your assessment and a detailed revision plan explaining what the author should change and why. Include a link to your posted GitHub Issue in your Homework 6 answer.

ANSWER: 1. The report I chose to use belongs to the GitHub user Bich-Yen. The question I selected is Question 5, which involves creating a short SKILL.md file for improving the formatting of STAT 432 homework plots. I think the work could be improved by providing a side-by-side comparison of Question 1's plots with and without using the skill. The question prompts us to explicitly call the skill to improve the plot from Question 1, then compare the original and revised versions. However, I only see one plot embedded within Question 1, and there is no additional plot located within Question 5. Therefore, I am unable to view the improvements or changes that the skill provided.

2. The code the user used to generate their Question 1 plot is:

```
matplot(k, cbind(squared_bias, variance, MSE_direct), type = "b",
        pch = c(16, 17, 15), lty = 1,
        col = c("darkorange", "steelblue", "black"),
        xlab = "Number of neighbors (k)", ylab = "Estimated error",
        main = "KNN bias-variance trade-off")
legend("topright", c("Squared bias", "Variance", "MSE"),
       col = c("darkorange", "steelblue", "black"),
       pch = c(16, 17, 15), lty = 1)
```

Additionally, the author says " I explicitly used the skill's rules to revise the Question 1 plot: the revised code labels both axes, uses a descriptive title, connects the three series, distinguishes them with colors and point shapes, and includes a legend. Compared with the original points-only plot, these changes make it easier to identify and compare the series without changing the calculations. I prefer the revised design because the labels and legend explain the plot more clearly. The final rendered appearance should be checked when the document is rendered."

This explanation is sufficient and follows the instructions of "Explicitly call it to improve the plot from Question 1, then compare the original and revised versions. Do you like the changes?" However, since I am unable to verify these improvements, it gives this area room for improvement. I can see the distinguished colors and shapes, but I do not have a baseline for comparison. This answer could be improved by displaying a before vs after plot of using the newly created skill. Then, there would be visual evidence and greater support for the author's claims.

3. To format the improved answer, I would keep the written explanation the author has already. However, I would leave the original plot in question 1 unchanged, and then include the improved plot in question 5. So, there report should essentially include the same plot twice, just different versions in different sections of the assignment. Then, I would make explicit references to the improvements the author is discussing. The code does not need to be changed, the user just needs to instruct its AI agent to not use the skill on the plot in Question 1 but do use it on the plot for Question 5.

Link to posted GitHub Issue: <https://github.com/Bich-Yen/stat432-fall2026/issues/2>